LAPORAN PRAKTIKUM

STRUKTUR DATA

“QUEUE”

disusun Oleh:

NAIRA RAMADHANI HALIL

2511533027

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T.

Asisten Praktikum:

ZAHRAN AZARIA ANVAYA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena berkat rahmat dan karunia-Nya, laporan praktikum dengan judul “QUEUE” dapat diselesaikan tepat waktu.

Laporan ini disusun untuk memenuhi salah satu tugas mata kuliah Praktikum Struktur Data, sekaligus sebagai sarana pembelajaran dalam memahami konsep dasar pemrograman khususnya Queue pada Bahasa Java.

Pada kesempatan ini, penulis menyampaikan terimakasih kepada:

1. Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu mata kuliah Praktikum Struktur Data yang telah memberikan bimbingan dan arahan.
2. Kak Zahran Azaria Anvaya selaku asisten praktikum kelas A yang telah membantu pelaksanaan praktikum.
3. Teman-teman mahasiswa yang telah membantu dan memberi dukungan serta berdiskusi bersama dalam menyelesaikan laporan ini.

Penulis menyadari bahwa laporan ini masih jauh dari kata sempurna. Oleh karena itu, krirtik dan saran yang membangun sangat diharapkan demi penyempurnaan laporan di masa mendatang.

Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan menambah wawasan mengenai pemrograman serta struktur data pada Java.

DAFTAR ISI

KATA PENGANTAR ..... **i**
DAFTAR ISI ..... **ii**
BAB I **1**
1.1 Latar Belakang 1
1.2 Tujuan Praktikum 1
1.3 Manfaat Praktikum 2
BAB II **3**
2.1 Langkah Kerja Pratikum 3
2.2 Analisis Hasil dan Pembahasan 11
BAB III **14**
3.1 Kesimpulan 14
DAFTAR PUSTAKA .....

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam praktikum ini, aku mulai dari hal yang cukup mendasar tapi ternyata "dalem" juga, yaitu struktur data **queue (antrian)**. Awalnya kelihatannya sederhana, cuma konsep masuk dari belakang, keluar dari depan. Tapi pas dicoba di kode, ternyata banyak hal kecil yang harus dipahami biar gak salah logika. Misalnya gimana cara nyimpen data, gimana ngatur posisi depan dan belakang, sampai gimana cara menampilkan isi antrian tanpa bikin program jadi kacau.

Di codingan 1 dan 4, aku lihat penggunaan queue dari Java langsung pakai LinkedList, jadi lebih praktis. Tapi di codingan 2 dan 3, queue dibuat manual pakai array, dan di situ baru kerasa "oh ternyata ini cara kerjanya di balik layar". Ada konsep seperti circular queue juga yang bikin aku harus mikir dua kali. Terus di codingan 5, ada tambahan stack buat reverse data queue, yang awalnya agak membingungkan, tapi setelah dipahami ternyata itu kombinasi struktur data yang menarik.

Jadi, praktikum ini bukan cuma tentang "pakai queue", tapi juga tentang memahami bagaimana queue itu bekerja, baik dari sisi penggunaan maupun implementasinya.

1.2 Tujuan Praktikum

- Memahami konsep dasar **queue (antrian)** dengan prinsip FIFO (First In First Out).
- Mengetahui cara kerja operasi dasar queue seperti enqueue, dequeue, peek, dan size.
- Membedakan implementasi queue menggunakan **library Java (LinkedList)** dan implementasi manual menggunakan **array**.
- Melatih pemahaman tentang pengelolaan indeks pada queue berbasis array (termasuk konsep circular queue).
- Memahami cara iterasi data dalam queue menggunakan *iterator*.
- Mengetahui bagaimana mengkombinasikan struktur data, seperti penggunaan **stack untuk membalik isi queue**.

1.3 Manfaat Praktikum

- Membantu memahami penerapan queue dalam kehidupan nyata, seperti sistem antrian atau pemrosesan data.
- Meningkatkan pemahaman logika pemrograman, terutama dalam pengelolaan data berurutan.
- Menambah kemampuan dalam menggunakan struktur data bawaan Java secara efisien.
- Memberikan pengalaman membuat struktur data secara manual sehingga lebih paham cara kerjanya.
- Melatih ketelitian dalam mengelola indeks dan alur program agar tidak terjadi error.
- Menambah wawasan tentang penggunaan lebih dari satu struktur data untuk menyelesaikan suatu masalah.

BAB II
PEMBAHASAN

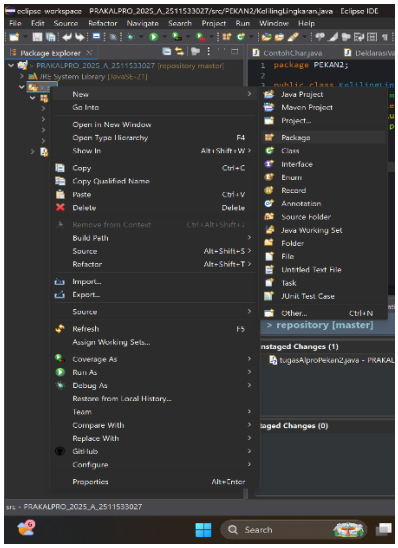
2.1 Langkah Kerja Pratikum

1. Persiapan Awal.

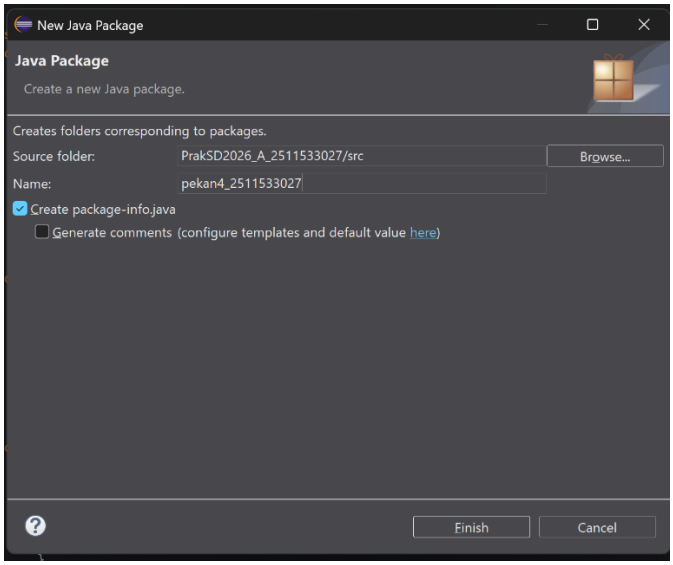
   a) Buka apl Eclipse IDE di alptop atau di komputer.

2. Membuat Package.

   a) Klik kanan pada folder **src**, lalu pilih **New**, terakhir klik **Package**.

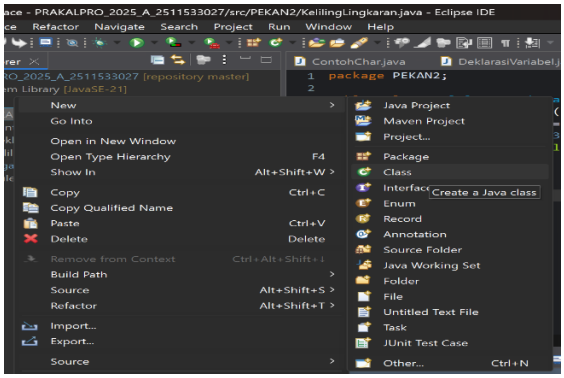


   b) Beri nama package **pekan4_NIM**.



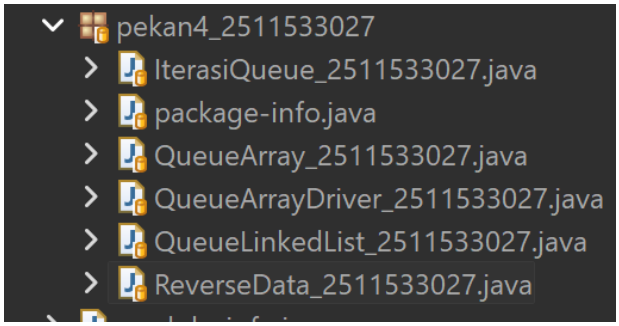
3. Membuat Class Baru untuk Setiap Percobaan.

   a) Klik kanan pada package **pekan4_NIM**, lalu pilih **New**, terakhir klik **Class**.



   b) Buat beberapa kelas sesuai percobaan.

   - IterasiQueue_NIM.java
   - QueueArray_NIM.java
   - QueueArrayDriver_NIM.java
   - QueueLinkedList_NIM.java
   - ReverseData_NIM.java



4. Menulis Kode Program.

   1. IterasiQueue_NIM.java

- Program ini digunakan untuk memahami cara melakukan iterasi pada struktur data queue.
- Queue bertipe data string digunakan untuk menyimpan beberapa kata.
- Beberapa elemen ditambahkan ke dalam queue menggunakan method add().
- Program menggunakan Iterator untuk mengakses setiap elemen dalam queue.
- Perulangan while digunakan untuk menampilkan isi queue satu per satu.
- Method hasNext() berfungsi untuk mengecek apakah masih ada elemen berikutnya.
- Method next() digunakan untuk mengambil elemen secara berurutan dari queue.
- Hasil akhir berupa tampilan seluruh isi queue dalam satu baris.

```java
package pekan4_2511533027;

import java.util.Iterator;

public class IterasiQueue_2511533027 {

    public static void main(String[] args) {
        Queue<String> q_3027 = new LinkedList<>();
        q_3027.add("Praktikum");
        q_3027.add("Struktur");
        q_3027.add("Data");
        q_3027.add("Dan");
        q_3027.add("Algoritma");
        Iterator<String> iterator = q_3027.iterator();
        while (iterator.hasNext()) {
            System.out.print(iterator.next() + " ");
        }
    }

}
```

2. QueueArray_NIM.java
   - Program ini digunakan untuk memahami implementasi queue secara manual menggunakan array.
   - Queue direpresentasikan dengan array serta variabel front, rear, size, dan capacity.
   - Konstruktor digunakan untuk menginisialisasi kapasitas dan kondisi awal queue.
   - Method isFull() digunakan untuk mengecek apakah queue sudah penuh.
   - Method isEmpty() digunakan untuk mengecek apakah queue kosong.

- Method enqueue() digunakan untuk menambahkan elemen ke dalam queue dengan konsep circular.
- Method dequeue() digunakan untuk menghapus elemen dari bagian depan queue.
- Method front() dan rear() digunakan untuk melihat elemen terdepan dan terakhir.
- Method display() digunakan untuk menampilkan isi queue.
- Program ini menunjukkan bagaimana queue bekerja di balik layar tanpa bantuan library.

```
package pekan4_2511533027;

public class QueueArray_2511533027 {
    int front_3027, rear_3027, size_3027;
    int capacity_3027;
    int array_3027[];

    public QueueArray_2511533027(int capacity) {
            this.capacity_3027 = capacity;
            front_3027 = this.size_3027 = 0;
            rear_3027 = capacity - 1;
            array_3027 = new int[this.capacity_3027];
    }

    boolean isFull_3027(QueueArray_2511533027 queue) {
            return (queue.size_3027 == queue.capacity_3027);
    }

    boolean isEmpty_3027(QueueArray_2511533027 queue) {
            return (queue.size_3027 == 0);
    }

    void enqueue_3027(int item) {
            if (isFull_3027(this))
                    return;
            this.rear_3027 = (this.rear_3027 + 1) % this.capacity_3027;
            this.array_3027[this.rear_3027] = item;
            this.size_3027 = this.size_3027 + 1;
```

```
            System.out.println(item + " enqueued to queue");
    }

    int dequeue_3027() {
            if (isEmpty_3027(this))
                    return Integer.MIN_VALUE;
            int item_3027 = this.array_3027[this.front_3027];
            this.front_3027 = (this.front_3027 + 1) % this.capacity_3027;
            this.size_3027 = this.size_3027 - 1;
            return item_3027;
    }

    int front_3027() {
            if (isEmpty_3027(this))
                    return Integer.MIN_VALUE;
            return this.array_3027[this.front_3027];
    }

    int rear_3027() {
            if (isEmpty_3027(this))
                    return Integer.MIN_VALUE;
            return this.array_3027[this.rear_3027];
    }

            // mencetak elemen antrian
    void display() {
            int i;
            if (front_3027 == rear_3027) {
                    System.out.printf("\nAntrian Kosong\n");
                    return;
            }

            //kunjungi dari belakang dan cetak
            for (i = front_3027; i < rear_3027; i++) {
                    System.out.printf(" %d <-- ", array_3027[i]);
            }
            return;
    }

}
```

3. QueueArrayDriver_NIM.java

- Program ini digunakan sebagai driver untuk menjalankan dan menguji class QueueArray.
- Objek queue dibuat dengan kapasitas tertentu.
- Program menampilkan elemen terdepan menggunakan method front().
- Beberapa elemen dimasukkan ke dalam queue menggunakan method enqueue().
- Program menampilkan elemen terakhir menggunakan method rear().
- Method display() digunakan untuk menampilkan seluruh isi queue.
- Method dequeue() digunakan untuk menghapus satu elemen dari queue.
- Setelah penghapusan, program kembali menampilkan kondisi queue terbaru.
- Hasil akhir menunjukkan perubahan isi queue sebelum dan sesudah penghapusan.

```
package pekan4_2511533027;

public class QueueArrayDriver_2511533027 {

    public static void main(String[] args) {
        QueueArray_2511533027 queue_3027 = new
QueueArray_2511533027(1000);
        queue_3027.enqueue_3027(10);
        queue_3027.enqueue_3027(20);
        queue_3027.enqueue_3027(30);
        queue_3027.enqueue_3027(40);
        System.out.println("Item di depan "+ queue_3027.front_3027());
        System.out.println("Item paling belakang " +
queue_3027.rear_3027());
        System.out.println("tampilan queue");
        queue_3027.display();
        System.out.println();
        System.out.println(queue_3027.dequeue_3027() + " dihapus dari
queue");
        System.out.println("Item di depan:" +
queue_3027.front_3027());
```

```
        System.out.println("Item dibelakang: " +
queue_3027.rear_3027());
        System.out.println("Tampilan queue setelah satu data
dihapus");
        queue_3027.display();
    }

}
```

4. QueueLinkedList_NIM.java
   - Program ini digunakan untuk memahami penggunaan queue dengan bantuan class LinkedList di Java.
   - Queue bertipe data Integer digunakan untuk menyimpan beberapa angka.
   - Elemen ditambahkan ke dalam queue menggunakan perulangan dan method add().
   - Isi queue ditampilkan secara langsung menggunakan System.out.println().
   - Method remove() digunakan untuk menghapus elemen terdepan dari queue.
   - Method peek() digunakan untuk melihat elemen paling depan tanpa menghapusnya.
   - Method size() digunakan untuk mengetahui jumlah elemen dalam queue.
   - Program ini menunjukkan penggunaan queue yang lebih praktis dibanding implementasi manual.

```
package pekan4_2511533027;

import java.util.LinkedList;
import java.util.Queue;

public class QueueLinkedList_2511533027 {

    public static void main(String[] args) {
        Queue<Integer> q_3027 = new LinkedList<>();
        // tambah elemen (0, 1, 2, 3, 4, 5) ke antrian
        for (int i = 0; i < 6; i++)
            q_3027.add(i);
        // menampilkan isi antrian.
         System.out.println("Elemen Antrian " + q_3027);
```

```java
        // untuk mengahpus kepala antrian.
        int hapus_3027 = q_3027.remove();
        System.out.println("Hapus Elemen = " + hapus_3027);
        System.out.println(q_3027);

        // untuk melihat antrian terdepan
        int depan_3027 = q_3027.peek();
        System.out.println("Kepala Antrian = " + depan_3027);

        int banyak_3027 = q_3027.size();
        System.out.println("Size Antrian = " + banyak_3027);
    }

}
```

5. ReverseData_NIM.java
    - Program ini digunakan untuk memahami cara membalik urutan data dalam queue.
    - Queue bertipe Integer digunakan untuk menyimpan beberapa elemen angka.
    - Stack digunakan sebagai struktur data tambahan untuk membantu proses pembalikan.
    - Elemen dari queue dipindahkan ke stack menggunakan method remove() dan push().
    - Proses ini membuat urutan elemen menjadi terbalik saat masuk ke dalam stack.
    - Elemen kemudian dikembalikan ke queue menggunakan method pop() dan add().
    - Setelah proses selesai, isi queue menjadi terbalik dari urutan awal.
    - Program ini menunjukkan kombinasi penggunaan queue dan stack dalam menyelesaikan masalah.

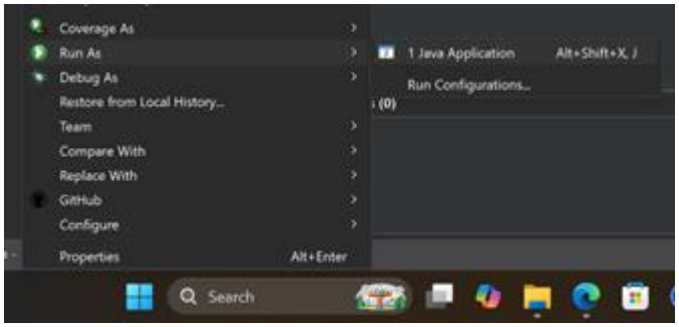
```java
package pekan4_2511533027;

import java.util.LinkedList;

public class ReverseData_2511533027 {
    public static void main(String[] args) {
        Queue<Integer> q_3027 = new LinkedList<Integer>();
        q_3027.add(1);
        q_3027.add(2);
        q_3027.add(3); // [1, 2, 3]
        System.out.println("Sebelum reverse" + q_3027);
        Stack<Integer> s_3027 = new Stack<Integer>();
        while (!q_3027.isEmpty()) { // Q -> S
            s_3027.push(q_3027.remove());
        }
        while (!s_3027.isEmpty()) { // S -> Q
            q_3027.add(s_3027.pop());
        }
        System.out.println("Sesudah reverse= " + q_3027); // [3, 2 ,1]
    }

}
```

5. Kompilasi dan Menjalankan Program.



2.2 Analisis Hasil dan Pembahasan

1. IterasiQueue_NIM.java
   a) Hasil: Program berhasil menampilkan seluruh elemen yang terdapat di dalam queue. Elemen-elemen yang telah dimasukkan sebelumnya dapat ditampilkan secara berurutan dalam satu baris. Proses iterasi berjalan dengan baik tanpa mengubah isi dari queue tersebut.
   b) Pembahasan: Pada program ini, struktur data Queue digunakan dengan bantuan class LinkedList. Untuk mengakses setiap elemen, digunakan Iterator yang memungkinkan traversal tanpa menghapus data. Method hasNext() berfungsi sebagai pengecek apakah masih ada elemen berikutnya, sedangkan next() digunakan untuk mengambil elemen tersebut. Pendekatan ini cukup efisien karena isi queue tetap utuh, sehingga cocok digunakan ketika hanya ingin membaca data tanpa memodifikasinya.
2. QueueArray_NIM.java
   a) Hasil: Program berhasil mengimplementasikan queue menggunakan array. Operasi seperti menambah elemen, menghapus elemen, serta melihat elemen

terdepan dan terakhir dapat berjalan dengan baik. Queue juga dapat menampilkan isi elemen sesuai urutan.

b) Pembahasan: Program ini mengimplementasikan queue secara manual dengan memanfaatkan array dan konsep circular queue. Variabel front dan rear digunakan untuk mengatur posisi elemen dalam antrian. Method enqueue() dan dequeue() bekerja dengan bantuan operasi modulo agar indeks dapat berputar kembali ke awal array. Hal ini membuat penggunaan memori lebih optimal. Program ini membantu memahami cara kerja queue secara lebih mendalam dibandingkan hanya menggunakan library.

3. QueueArrayDriver_NIM.java

a) Hasil: Program berhasil menjalankan operasi queue seperti penambahan data, penghapusan data, serta menampilkan isi queue sebelum dan sesudah perubahan. Elemen terdepan dan terakhir juga dapat ditampilkan dengan benar.

b) Pembahasan: Program ini berperan sebagai penguji dari class QueueArray. Setiap method yang telah dibuat diuji satu per satu untuk memastikan fungsinya berjalan sesuai harapan. Dengan adanya driver ini, dapat terlihat perubahan kondisi queue setelah dilakukan operasi tertentu, seperti enqueue() dan dequeue(). Program ini juga membantu dalam memahami alur kerja queue secara lebih nyata melalui output yang dihasilkan.

4. QueueLinkedList_NIM.java

a) Hasil: Program berhasil menambahkan beberapa elemen ke dalam queue, menampilkan isi queue, menghapus elemen terdepan, serta menampilkan ukuran queue. Semua operasi berjalan dengan baik menggunakan library yang tersedia.

b) Pembahasan: Program ini menggunakan implementasi queue berbasis LinkedList yang sudah disediakan oleh Java, sehingga lebih sederhana dan tidak perlu mengatur indeks secara manual. Method seperti add(), remove(), peek(), dan size() mempermudah pengolahan data dalam queue. Dibandingkan dengan queue berbasis array, pendekatan ini lebih fleksibel dan mudah digunakan, meskipun pengguna tidak melihat langsung mekanisme internalnya.

5. ReverseData_NIM.java

a) Hasil: Program berhasil membalik urutan elemen dalam queue. Elemen yang awalnya berurutan dari depan ke belakang berubah menjadi sebaliknya setelah proses selesai.

b) Pembahasan: Pada program ini, digunakan dua struktur data yaitu queue dan stack. Queue digunakan sebagai penyimpan data utama, sedangkan stack berfungsi sebagai alat bantu untuk membalik urutan. Proses dilakukan dengan memindahkan seluruh elemen dari queue ke stack, kemudian mengembalikannya lagi ke queue. Karena stack menggunakan prinsip LIFO, urutan elemen otomatis menjadi terbalik. Program ini menunjukkan bagaimana dua struktur data dapat dikombinasikan untuk menyelesaikan suatu permasalahan secara efektif.

BAB III
PENUTUP

3.1 Kesimpulan

Dari praktikum yang telah dilakukan, dapat dipahami bahwa struktur data **queue** bekerja dengan prinsip FIFO (First In First Out), yaitu data yang pertama masuk akan menjadi data yang pertama keluar. Konsep ini terlihat jelas melalui berbagai operasi seperti enqueue, dequeue, serta pengambilan elemen terdepan dan terakhir.

Selain itu, queue dapat diimplementasikan dengan dua cara, yaitu menggunakan library Java seperti LinkedList yang lebih praktis, dan menggunakan array yang dibuat secara manual. Implementasi manual memberikan pemahaman yang lebih mendalam, terutama dalam pengelolaan indeks seperti front dan rear, serta penerapan konsep circular queue.

Praktikum ini juga menunjukkan bahwa queue dapat dikombinasikan dengan struktur data lain, seperti stack, untuk menyelesaikan masalah tertentu seperti membalik urutan data. Secara keseluruhan, kegiatan ini membantu meningkatkan pemahaman konsep, logika pemrograman, serta cara berpikir dalam menyelesaikan masalah menggunakan struktur data.

DAFTAR PUSTAKA

[1] Wahyudi, *Materi Struktur Data: Queue*. Padang: PPT Perkuliahan, 2026.

[5] Oracle, "Java Platform SE Documentation: Queue Interface," 2024. [Online]. Available: https://docs.oracle.com

LAPORAN TUGAS PRAKTIKUM

STRUKTUR DATA

“QUEUE”

disusun Oleh:

NAIRA RAMADHANI HALIL

2511533027

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T.

Asisten Praktikum:

ZAHRAN AZARIA ANVAYA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Dengan memanjatkan puji syukur kehadirat Allah SWT, atas rahmat dan karunia-Nya sehingga laporan tugas praktikum Struktur Data ini dapat diselesaikan dengan baik. Shalawat serta salam semoga selalu tercurah kepada Nabi Muhammad SAW.

Laporan tugas ini disusun untuk memenuhi tugas praktikum Struktur Data dengan topik Queue menggunakan studi kasus Website dan Browser. Pada praktikum ini, penulis membuat program Java yang dapat mengelola data Website seperti menambah, menampilkan, dan menghapus data.

Penulis mengucapkan terima kasih kepada dosen pengampu dan asisten praktikum atas bimbingannya. Penulis juga menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran sangat diharapkan.

DAFTAR ISI

KATA PENGANTAR ..... **i**
DAFTAR ISI ..... **ii**
BAB I **3**
1.1 Latar Belakang 3
1.2 Tujuan 3
1.3 Manfaat 3
1.4 Instruksi Tugas 3
BAB II **6**
2.1 Kelas AntrianLoket_NIM 6
BAB III **9**
3.1 Kesimpulan 9
DAFTAR PUSTAKA .....

BAB I
PENDAHULUAN

1.1 Latar Belakang

Queue merupakan salah satu struktur data yang bekerja dengan prinsip **First In First Out (FIFO)**, yaitu data yang pertama masuk akan menjadi data pertama yang keluar. Konsep ini sering digunakan dalam kehidupan sehari-hari, salah satunya pada sistem antrian, seperti antrian di loket pelayanan. Setiap pelanggan yang datang akan masuk ke dalam antrian, kemudian dilayani sesuai urutan kedatangan.

Pada tugas ini, konsep queue diterapkan dalam simulasi **antrian pelanggan di loket pelayanan**, di mana setiap pelanggan yang datang akan ditambahkan ke dalam antrian, dan pelanggan yang berada di posisi paling depan akan dilayani terlebih dahulu. Program ini juga dilengkapi dengan fitur untuk menampilkan isi antrian serta membalik urutan antrian menggunakan struktur data tambahan.

Melalui program ini, konsep queue dapat dipahami secara lebih nyata, tidak hanya secara teori tetapi juga melalui implementasi langsung dalam bentuk program sederhana.

1.2 Tujuan

Tujuan:

1. Memahami konsep dasar struktur data queue dan prinsip FIFO.
2. Menerapkan operasi queue seperti enqueue, dequeue, dan display pada Java.
3. Mengimplementasikan queue menggunakan array sebagai media penyimpanan data.
4. Membuat simulasi antrian pelanggan di loket pelayanan.
5. Menerapkan proses pembalikan data antrian (reverse) menggunakan struktur data tambahan.

1.3 Manfaat

Manfaat:

1. Membantu memahami cara kerja queue secara praktis.
2. Melatih penggunaan struktur data queue dalam pemrograman Java.
3. Mempermudah pemahaman konsep FIFO melalui simulasi antrian.
4. Menambah kemampuan dalam mengelola data menggunakan array.
5. Menjadi dasar untuk penerapan queue pada kasus lain yang lebih kompleks.

1.4 Instruksi Tugas

1. Nama Kelas

Gunakan akhiran NIM lengkap

Contoh:

- AntrianLoket_2311532008

2. Nama File Java

   Harus sama dengan nama class

   Contoh:

   - AntrianLoket_2311532008.java

3. Penamaan Variabel
   - front → indeks depan
   - rear → indeks belakang
   - max → kapasitas array
   - queue[] → array penyimpan data

4. Struktur Tugas

   a) Deklarasi dan Inisialisasi
      - Array untuk menyimpan data antrian
      - Variabel front dan rear
      - Kapasitas maksimal queue

   b) Method yang Wajib Ada
      1. enqueue()
         - Menambahkan data ke belakang antrian
         - Jika penuh → tampilkan pesan
      2. dequeue()
         - Menghapus data dari depan antrian
         - Jika kosong → tampilkan pesan
      3. display()
         - Menampilkan seluruh isi antrian
      4. isEmpty()
         - Mengembalikan nilai true jika kosong
      5. isFull()
         - Mengembalikan nilai true jika penuh
      6. Reverse()
         - Membalik seluruh isi antrian

5. Contoh Output

=== PROGRAM ANTRIAN LOKET ===

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar

Pilih menu: 1

Masukkan nama pelanggan: Andi

Data berhasil ditambahkan ke antrian

Pilih menu: 1

Masukkan nama pelanggan: Budi

Data berhasil ditambahkan ke antrian

Pilih menu: 3

Isi antrian: 1. Andi 2. Budi

Pilih menu: 4

Isi antrian: 1. Budi 2. Andi

Pilih menu: 2

Budi telah dilayani

Pilih menu: 3

Isi antrian: 1. Andi

Pilih menu: 5

Program selesai

BAB II
PEMBAHASAN

2.1 Kelas AntrianLoket_NIM

Kelas **AntrianLoket_2511533027** digunakan untuk merepresentasikan struktur data queue (antrian) pada program. Di dalam kelas ini terdapat atribut front, rear, dan max untuk mengatur posisi serta kapasitas antrian, serta array queue[] untuk menyimpan data pelanggan.

Kelas ini memiliki method seperti enqueue() untuk menambah data, dequeue() untuk menghapus data, dan display() untuk menampilkan isi antrian. Selain itu, terdapat method isEmpty() dan isFull() untuk mengecek kondisi antrian, serta reverse() untuk membalik urutan data.

Kelas ini menjadi bagian utama dalam program karena seluruh proses pengolahan antrian dilakukan di dalamnya.

```java
package pekan4_2511533027;

import java.util.Scanner;
import java.util.Stack;

public class AntrianLoket_2511533027 {
    int front_3027, rear_3027, max_3027;
    String queue_3027[];

    // konstruktor
    public AntrianLoket_2511533027(int max_3027) {
        this.max_3027 = max_3027;
        queue_3027 = new String[max_3027];
        front_3027 = -1;
        rear_3027 = -1;
    }

    // cek kondisi antrian kosong
    boolean isEmpty_3027() {
        return (front_3027 == -1);
    }

    // cek antrian penuh
    boolean isFull_3027() {
        return (rear_3027 == - 1);
    }

    // enqueue (tambah pelanggan ke antrian)
    void enqueue_3027(String data) {
            if (isEmpty_3027()) {
                front_3027 = 0;
            }
            rear_3027++;
            queue_3027[rear_3027] = data;
             System.out.println("Data berhasil ditambahkan ke antrian");

    }
```

```java
// dequeue (hapus data pelanggan dari antrian)
void dequeue_3027() {
    if (isEmpty_3027()) {
        System.out.println("Antrian Kosong!");
    } else {
        System.out.println(queue_3027[front_3027] + " Telah Dilayani");
        if (front_3027 == rear_3027) {
            front_3027 = rear_3027 = -1;
        } else {
            front_3027++;
        }
    }
}

// tampilkan antrian pelanggan
void display_3027() {
    if (isEmpty_3027()) {
        System.out.println("Antrian Kosong!");
    } else {
        System.out.println("Isi Antrian:");
        int no_3027 = 1;
        for (int i = front_3027; i <= rear_3027; i++) {
            System.out.println(no_3027 + "." + queue_3027[i]);
            no_3027++;
        }
    }
}
```

```java
// reverse data antrian pelanggan
void reverse_3027() {
    if (isEmpty_3027()) {
        System.out.println("Antrian Kosong!");
        return;
    }

    Stack<String> stack_3027 = new Stack<>();
    for (int i = front_3027; i <= rear_3027; i++) {
        stack_3027.push(queue_3027[i]); // Q -> stack
    }

    int i = front_3027;
    while (!stack_3027.isEmpty()) {
        queue_3027[i] = stack_3027.pop();
        i++; // stack -> Q
    }
    System.out.println("Antrian Berhasil Dibalik");
    // tampilkan setelah dibalik
    System.out.println("Isi Antrian:");
    display_3027();
}
```

```java
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        AntrianLoket_2511533027 antrian_3027 = new AntrianLoket_2511533027(10);
        int pilih_3027;
        do {
             System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
             System.out.println("1. Tambah Antrian");
             System.out.println("2. Hapus Antrian");
             System.out.println("3. Tampilkan Antrian");
             System.out.println("4. Reverse");
             System.out.println("5. Keluar");
             System.out.print("Pilih Menu: ");
             pilih_3027 = input.nextInt();
             input.nextLine();

             switch (pilih_3027) {
             case 1:
                 System.out.print("Masukkan Nama Pelanggan: ");
                 String nama_3027 = input.nextLine();
                 antrian_3027.enqueue_3027(nama_3027);
                 break;

             case 2:
                 antrian_3027.dequeue_3027();
                 break;

             case 3:
                 antrian_3027.display_3027();
                 break;

             case 4:
                 antrian_3027.reverse_3027();
                 break;

             case 5:
                 System.out.println("Program Selesai.");
                 break;

            default:
                 System.out.println("Pilihan Tidak Valid.");
             }
```

```java
        } while (pilih_3027 != 5);

        input.close();
    }

}
```

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum dan tugas yang telah dilakukan, dapat dipahami bahwa struktur data **queue** bekerja dengan prinsip FIFO (First In First Out), di mana data yang masuk lebih dulu akan diproses terlebih dahulu. Konsep ini sangat relevan dengan sistem antrian di kehidupan nyata, seperti pada pelayanan loket.

Melalui pembuatan program, queue dapat diimplementasikan menggunakan array dengan memanfaatkan variabel front dan rear untuk mengatur posisi data. Selain itu, operasi dasar seperti enqueue, dequeue, dan display dapat berjalan dengan baik, sehingga alur antrian dapat dikelola secara terstruktur.

Tidak hanya itu, program juga menunjukkan bahwa queue dapat dikembangkan lebih lanjut, misalnya dengan fitur reverse menggunakan bantuan stack. Hal ini memberikan pemahaman bahwa struktur data dapat saling dikombinasikan untuk menyelesaikan permasalahan dengan lebih efektif.

DAFTAR PUSTAKA

[1] Wahyudi, *Materi Struktur Data: Queue*. Padang: PPT Perkuliahan, 2026.

[5] Oracle, "Java Platform SE Documentation: Queue Interface," 2024. [Online]. Available: https://docs.oracle.com